

# JSON (Local JSON Validation Engine)

Milenko Burlica

28 July, 2026

# Table of Contents

|     |                                                                                                                                                               |   |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------|---|
| 1   |  Engine-spezifische Parameter .....                                          | 4 |
| 2   |  Das Sequenzielle Validierungs-Prinzip (Accumulated Assertion Logging) ..... | 5 |
| 3   |  Unterstützte Operationen (op) .....                                         | 6 |
| 3.1 | 1. Operation: ASSERT (Multi-Pfad-Validierung) .....                                                                                                           | 6 |
| 4   |  Der erweiterte JSONPath-Parser (Gson-Evaluator) .....                       | 7 |
| 4.1 | Unterstützte JSONPath-Syntaxen: .....                                                                                                                         | 7 |
| 5   |  Konsole-Logging & Live-Ausgabe (Log Alignment) .....                        | 8 |
| 5.1 | Konsolen-Ausgabe bei erfolgreichem Testlauf: .....                                                                                                            | 8 |
| 5.2 | Konsolen-Ausgabe und Runtime-Exception bei Fehlern: .....                                                                                                     | 8 |

Die **JSON (Local JSON Validation Engine)** dient zur automatisierten Verifizierung und Validierung von Inhalten in lokal gespeicherten JSON-Dateien. Sie reagiert dediziert auf den Testschritt-Typ `type: JSON`.

Im Gegensatz zu den Netzwerk-Engines (REST/SOAP) führt die JSON-Engine keine HTTP-Aufrufe aus, sondern analysiert bestehende JSON-Strukturen (z. B. exportierte Konfigurationen, Log-Dateien oder lokal zwischengespeicherte Payloads) direkt auf dem Dateisystem.

# 1 Engine-spezifische Parameter

Neben den Standardfeldern besitzt die JSON-Engine spezifische Steuerungs-Parameter im YAML:

- **file** (String, Pflichtfeld): Der absolute oder relative Pfad zur JSON-Zielcodedatei auf dem Dateisystem (z. B. `src/test/resources/payloads/config.json`).
- **parameters** (Map, Pflichtfeld): Eine strukturierte Liste von sequenziellen Schlüssel-Wert-Paaren ( `pathX` und `valueX` ), die die zu prüfenden JSONPath-Ausdrücke und deren erwartete Werte definieren.

## 2 Das Sequenzielle Validierungs-Prinzip (Accumulated Assertion Logging)

Die JSON-Engine unterscheidet sich in einem wesentlichen Architektur-Merkmal von anderen Engines: **Sie nutzt kein striktes Fail-Fast während der Evaluierung einzelner Felder.**

Wenn ein Testschritt mehrere Pfade überprüft, bricht die Engine nicht beim ersten Fehler ab. Stattdessen läuft der Validierungs-Zyklus vollständig bis zum letzten definierten Pfad durch:

1. Alle erfolgreichen Prüfungen werden sofort mit einem grünen `[PASSED]` im Log ausgegeben.
2. Alle fehlgeschlagenen Prüfungen werden sofort mit einem roten `[ERROR]` und dem tatsächlichen vs. erwarteten Wert protokolliert.
3. Die Fehler werden im Hintergrund in einem `StringBuilder` gesammelt.
4. Erst **nachdem alle Pfade evaluiert wurden**, wirft die Engine bei Vorhandensein von Fehlern eine zusammenfassende `RuntimeException` (Error Summary), die den gesamten Testschritt offiziell fehlschlagen lässt.

## 3 Unterstützte Operationen (op)

### 3.1 1. Operation: ASSERT (Multi-Pfad-Validierung)

Überprüft eine unbegrenzte Anzahl von JSON-Pfaden innerhalb der angegebenen Datei. Die Parameter müssen zwingend im fortlaufenden Format `path1 / value1` , `path2 / value2` angegeben werden.

#### Unterstützte Matching-Typen:

- **Standard-Match:** Wenn der erwartete Wert ein normaler Text ist, wird ein exakter String-Vergleich ( `.equals()` ) durchgeführt.
- **Wildcard-Match (Regex):** Enthält der erwartete Wert ein Asterisk-Zeichen ( `*` ), schaltet die Engine automatisch auf RegEx-Validierung um ( `.matches()` ).

#### YAML

```
- type: JSON
  op: ASSERT
  file: src/test/resources/logs/app_output.json
  parameters:
    path1: "$.logs[0].level"
    value1: "ERROR"
    path2: "$.logs[0]['event.code']"
    value2: "400"
    path3: "$.logs[0].message"
    value3: "User * failed to login"    # Wildcard-Match via Regex
```

## 4 Der erweiterte JSONPath-Parser (Gson-Evaluator)

Da die Engine intern auf Google-Gson basiert, nutzt sie einen speziell optimierten, robusten Token-Parser für JSONPath-Ausdrücke. Der Einstiegspunkt `$` ist optional (wird automatisch bereinigt).

Der Parser zerlegt den Pfad in atomare Tokens und unterstützt komplexe, tief verschachtelte Strukturen ohne syntaktische Einschränkungen:

### 4.1 Unterstützte JSONPath-Syntaxen:

1. **Standard-Objektnotation:** Punkt-getrennte Felder für flache Hierarchien.
  - *Beispiel:* `logs.level` oder `$.status.code`
2. **Unbegrenzt verschachtelte Arrays:** Direkt aufeinanderfolgende Index-Klammern für mehrdimensionale Arrays.
  - *Beispiel:* `logs[0][10][2]`
3. **Komplexe Map-Schlüssel (Bracket Notation):** Ermöglicht das Auslesen von JSON-Keys, die Sonderzeichen, Punkte oder reine Zahlen enthalten und über die Standard-Punktnotation abstürzen würden.
  - *Beispiel:* `['event.code']` oder `["10"]`

## 5 Konsole-Logging & Live-Ausgabe (Log Alignment)

Das Logging-System liefert eine transparente Echtzeit-Übersicht aller überprüften Pfade innerhalb des Testschritts:

### 5.1 Konsolen-Ausgabe bei erfolgreichem Testlauf:

Plaintext

```
Checking JSON assertions for file: src/test/resources/logs/app_output.json
[PASSED] logs[0].level == "ERROR"
[PASSED] logs[0]['event.code'] == "400"
[PASSED] logs[0].message == "User admin failed to login"
>>> ALL JSON ASSERTIONS PASSED
```

### 5.2 Konsolen-Ausgabe und Runtime-Exception bei Fehlern:

Plaintext

```
Checking JSON assertions for file: src/test/resources/logs/app_output.json
[PASSED] logs[0].level == "ERROR"
[ERROR] logs[0]['event.code'] == "200", expected "400"
[ERROR] logs[0].invalid_field -> Evaluation failed: Indeks niza [5] je van
granica. Veličina niza: 2
```

#### Resultierende Fehlermeldung (Exception Report):

Plaintext

```
RuntimeException: JSON Assertion failed for file: src/test/resources/logs/
app_output.json
- Path [logs[0]['event.code']] failed! Expected: [400], Actual: [200]
- Path [logs[0].invalid_field] crashed during evaluation! Reason: Indeks niza [5]
je van granica. Veličina niza: 2
```